

Chapter 0

The C toolbox

Operating-systems code is C (or, in SWEB, a careful subset of C++) with no safety net: no garbage collector, no bounds checks, no exceptions. Most bugs you will hunt in A1 are pointer and lifetime bugs that corrupt memory silently and crash much later, somewhere else. This chapter is the minimum toolkit: how to build with the right flags, how to let the compiler’s sanitizers find those bugs for you, and the three C ideas every later chapter relies on — pointers, lifetimes, and function pointers.

0.1 Toolchain: from source to a running program

```
gcc -std=gnu17 -Wall -Wextra -g -O1 -fsanitize=address,undefined prog.c -o prog
```

-Wall -Wextra turn on the warnings that catch real bugs (unused results, sign mix-ups, missing returns). Treat a warning as a bug report you have not read yet. The course modules also turn a few warnings into errors, e.g. **-Werror=implicit-function-declaration**: calling a function without a prototype silently truncates pointers to `int` on 64-bit machines.

-g debug information: line numbers in sanitizer reports and in `gdb`.

-O0/-O1/-O2 optimisation. Race bugs and missing `volatiles` often appear only at **-O2**, because the compiler then keeps values in registers and reorders memory accesses. The modules therefore test both a sanitizer build and an optimised build.

`make` rebuilds only what changed. Every module has a `Makefile` with the targets `run` (examples) and `test` (assignment). `make test IMPL=../../solutions/NN-...` builds the tests against the reference solution instead of your own files.

0.2 Sanitizers: let the compiler find your bugs

A sanitizer is compiler instrumentation plus a run-time library. The instrumented program checks every memory access (or every thread interaction) and prints a report at the *first* violation — not at the crash three functions later.

Sanitizer	Flag	Finds
AddressSanitizer	<code>-fsanitize=address</code>	out-of-bounds accesses (heap, stack, globals), use after free, use after return, double free, leaks
UndefinedBehaviorSanitizer	<code>-fsanitize=undefined</code>	signed overflow, bad shifts, misaligned pointers, NULL dereference, out-of-range enum values
ThreadSanitizer	<code>-fsanitize=thread</code>	data races (chapter 3), lock-order inversions (chapter 6)

Reading a report: the first lines name the *kind* of error (`heap-use-after-free`, `stack-buffer-overflow`, ...) and the access; then come three stack traces — where the bad access happened, where the memory was allocated, and where it was freed. Start at the first frame that is in your code.

ASan and TSan cannot be combined (they instrument memory differently), and ThreadSanitizer fails on recent kernels with high address-space randomisation (“*unexpected memory mapping*”). The modules’ test runner starts TSan binaries under `setarch -R`, which disables randomisation for that one process.

0.3 Pointers

A pointer is an address plus a type. The type says how many bytes `*p` reads and how far `p + 1` moves.

```
int a[4] = {1, 2, 3, 4};
int *p = a;           /* an array decays to a pointer to its first element */
p + 2;                /* address of a[2]: moves by 2 * sizeof(int) bytes */
*(p + 2) == p[2];     /* identical: indexing is pointer arithmetic */
sizeof a;             /* 16: the array; sizeof p would be 8: the pointer */
```

Out-parameters. C returns one value. A function that must produce more writes through pointers the caller passes in — `pthread_create(&tid, ...)` stores the new thread’s id in the caller’s variable, and every system call that reads data does the same with a buffer.

void *. A pointer to “something”. It converts to and from any object pointer without a cast in C, but cannot be dereferenced or used in arithmetic until it is converted back to a real type. Generic code (`qsort`, `pthread_create`, a thread library’s argument) passes `void *` and trusts the caller to know the real type.

Integers as pointers. Kernels constantly convert between pointers and integers: a system call receives user pointers as plain numbers in registers (SWEb: `size_t`, or its `pointer` type). `uintptr_t/size_t` are wide enough; `int` is not. This is also why a *number* chosen by a user must be validated before the kernel treats it as an address (chapter 7).

0.4 Lifetimes: stack, heap, static

Storage	Lives	Typical bug
stack (locals)	until the function returns	returning or storing the address of a local; a thread using a pointer to its creator's local after the creator returned
heap (<code>malloc</code>)	until <code>free</code>	use after free, double free, leak (never freed)
static (globals, <code>static</code>)	the whole program	shared between all threads: needs synchronisation (chapter 3)

The stack is the dangerous one for thread code: a loop that passes `&i` to every new thread passes the *same* address, which keeps changing — and may no longer exist when the thread reads it (module 02, `bug1_loop_index.c` and `bug2_dead_arguments.c`).

```
int *broken(void) {
    int x = 42;
    return &x;      /* x dies here: the caller gets a dangling pointer */
}
```

AddressSanitizer reports this as `stack-use-after-return` (with `detect_stack_use_after_return=1`, which the modules set).

0.5 Function pointers and callbacks

A function's name, used without parentheses, is its address. A function pointer stores such an address and calls through it:

```
int twice(int x) { return 2 * x; }
int (*f)(int) = twice;      /* pointer to function taking int, returning int */
f(21);                      /* 42 */
```

The callback pattern: function + context. A library that calls back into your code cannot know what data your code needs, so it takes a function pointer *and* an opaque `void *` that it hands back unchanged:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

`start_routine` is “what to run”, `arg` is “with what”. The library never looks inside `arg` — which is also why the SWEB kernel must *not* validate it as a pointer (chapter 7): it may just as well be the number 7 cast to `void *`. Returning a value works the same way backwards: `start_routine` returns a `void *` that `pthread_join` hands to the joiner.

In SWEB

SWEB is written in C++, but kernel C++: no exceptions, no standard library (it brings its own `ustl` containers), fixed-width types (`uint32`, `size_t`, `pointer` for addresses). User programs in `userspace/tests/` are plain C against SWEB's own small `libc` (`userspace/libc`); the `pthread_*` functions there are empty stubs you will fill in.

Key points

- Build with warnings on, and test with sanitizers *and* with optimisation.
- Read a sanitizer report from the top: kind of error, bad access, allocation site, free site.
- Know where every object lives and how long; never hand a pointer to a local to code that outlives the function.
- Callbacks come as *function pointer + opaque context*; the library never interprets the context.